

The Max Flow Problem

Augmenting path method

David Pisinger

`pisinger@man.dtu.dk`

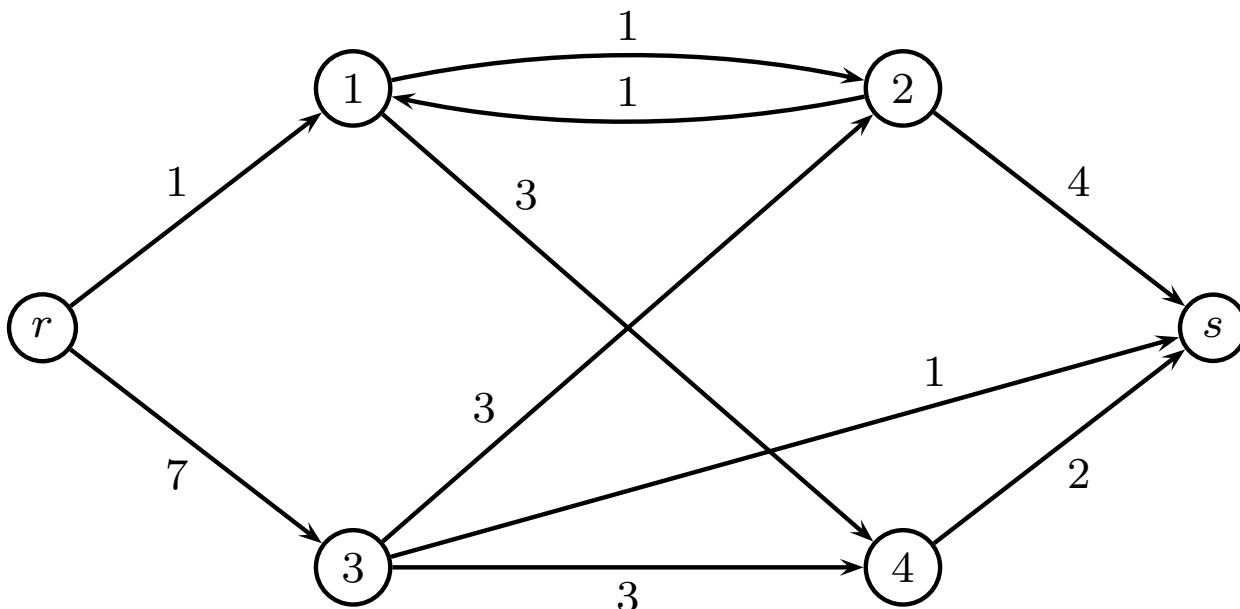
DTU Management

Technical University of Denmark

© Copyright David Pisinger. May not be distributed

The Max Flow Problem

- We consider a **directed graph** $G = (V, E)$ which for each e has a **capacity** $u_e \in \mathbb{R}_+$.
- Furthermore two “special” vertices r and s are given; these are called resp. the **source** and the **sink**.
- Aim is to find the maximum flow from the source to the sink in the graph while respecting the capacities of the edges.



Flow

● A **flow** in G is a function $x : E \rightarrow \mathbb{R}_+$ satisfying:

$$(1) \quad \forall i \in V \setminus \{r, s\} : \sum_{(j,i) \in E} x_{ji} - \sum_{(i,j) \in E} x_{ij} = 0$$

$$(2) \quad \forall (i, j) \in E : 0 \leq x_{ij} \leq u_{ij}$$

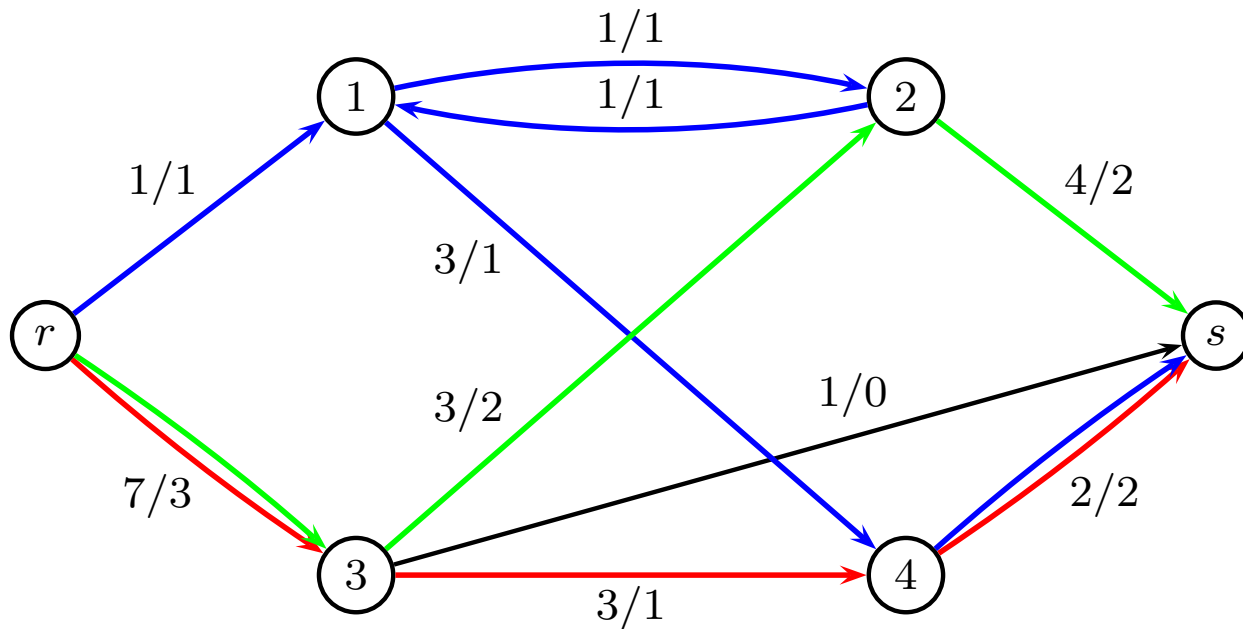
● For a flow we define: $f_x(i) = \sum_{(j,i) \in E} x_{ji} - \sum_{(i,j) \in E} x_{ij}$

● $f_x(i)$ is the **net flow into i** or the **excess for x in i** .

● $f_x(s)$ is called the **value of x** .

Example with a flow

Here we have added a flow of value 4 to our graph.



Notation: u_{ij}/x_{ij}

MIP Model of Max Flow

$$\begin{array}{ll}\max & f_x(s) \\ \text{s.t.} & f_x(i) = 0 \quad i \in V \setminus \{r, s\} \\ & 0 \leq x_e \leq u_e \quad e \in E \\ & x_e \in \mathbb{Z}_+ \quad e \in E\end{array}$$

where $f_x(i) = \sum_{(j,i) \in E} x_{ji} - \sum_{(i,j) \in E} x_{ij}$

Can be solved with LP-solver

Totally unimodular

	x_{e_1}	x_{e_2}	...	...	x_{e_m}		
1	-1		1	...		=	0
2		-1		...		=	0
.	1			...	1	=	.
.		1		...		=	.
n			-1	...	-1	=	0
e_1	1					≤	u_1
e_2		1				≤	u_2
.			1			≤	.
.						≤	.
e_m					1	≤	u_m

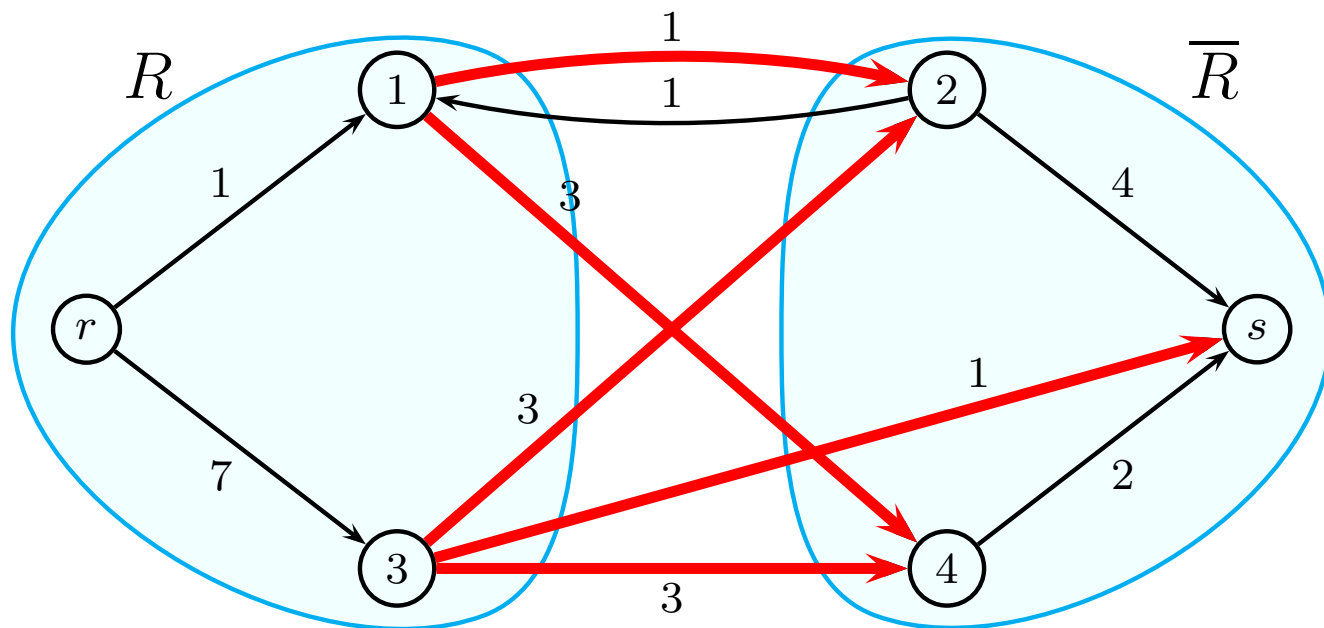
(No flow conservation constraints for node r and s)

Max Flow and Cuts I

- We now consider $R \subset V$. $\delta(R)$ is the set of edges incident from a vertex in R to a vertex in $\overline{R}$. $\delta(R)$ is also called the cut **generated** by R :

$$\delta(R) = \{(i, j) \in E : i \in R, j \in \overline{R}\}$$

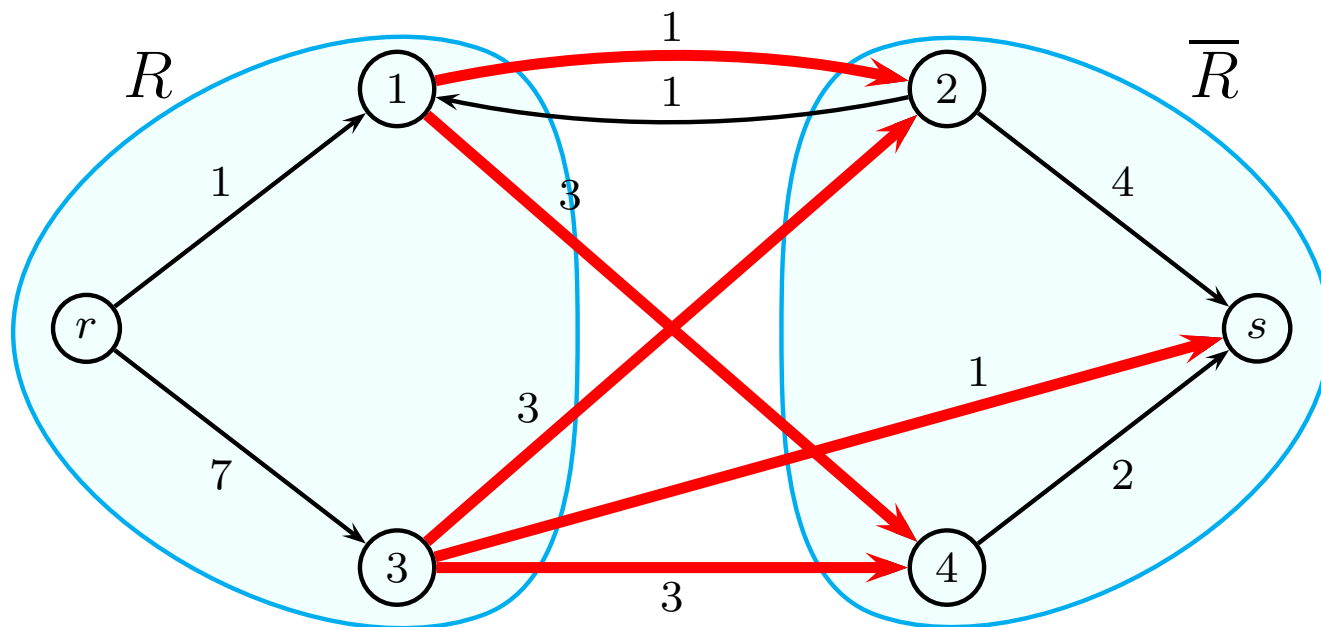
- Note the difference to the cut in an undirected graph. In a directed graph the cut only consists of arcs going from R to $\overline{R}$ and not the opposite direction.



Max Flow and Cuts II

- In an undirected graph we have $\delta(R) = \delta(\overline{R})$, this does not hold here.
- R is an **r,s-cut** if $r \in R$ and $s \notin R$.
- The **capacity** of the cut R is defined as:

$$u(\delta(R)) = \sum_{(i,j) \in E, i \in R, j \in \overline{R}} u_{ij}$$



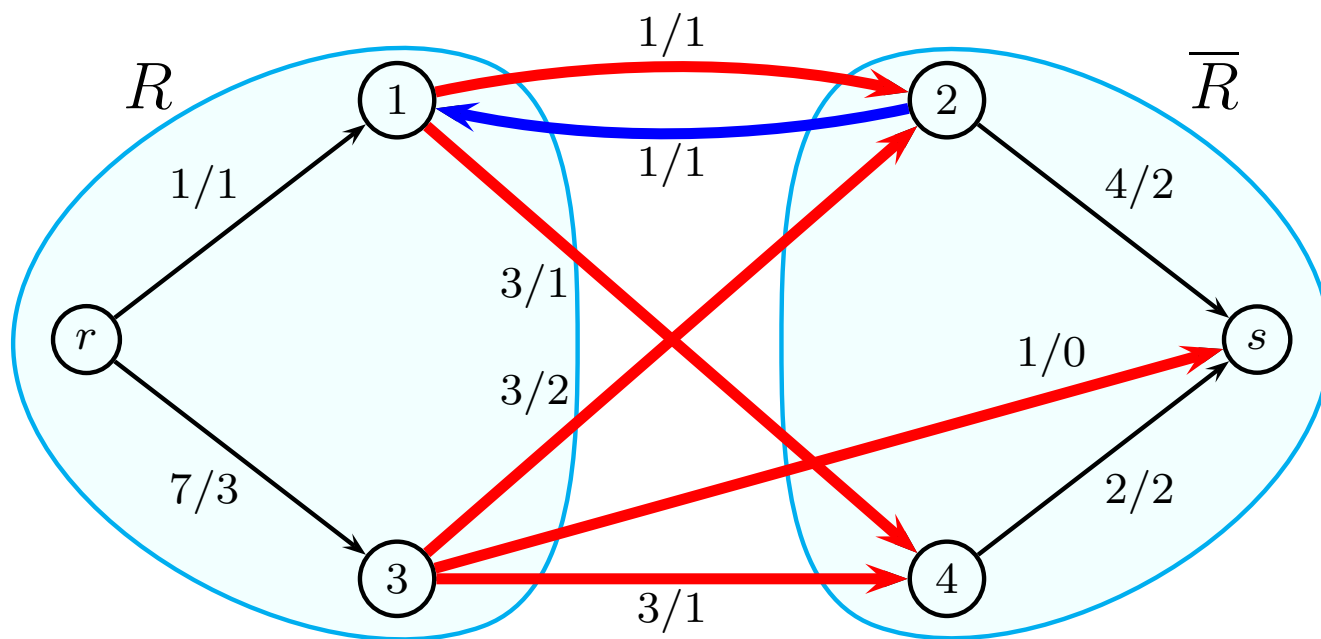
Relationship between flows and cuts

- **Proposition:** For any r, s -cut $\delta(R)$ and any flow x we have:

$$x(\delta(R)) - x(\delta(\bar{R})) = f_x(s)$$

- **Corollary:** For any feasible flow x and any r, s -cut $\delta(R)$, we have:

$$f_x(s) = x(\delta(R)) - x(\delta(\bar{R})) \leq x(\delta(R)) \leq u(\delta(R))$$



The Residual graph

- Suppose that x is a flow in G . The **residual graph** shows *how flow excess can be moved* in G given that the flow x is already present.
- The **residual graph** G_x for G wrt. x is defined by:

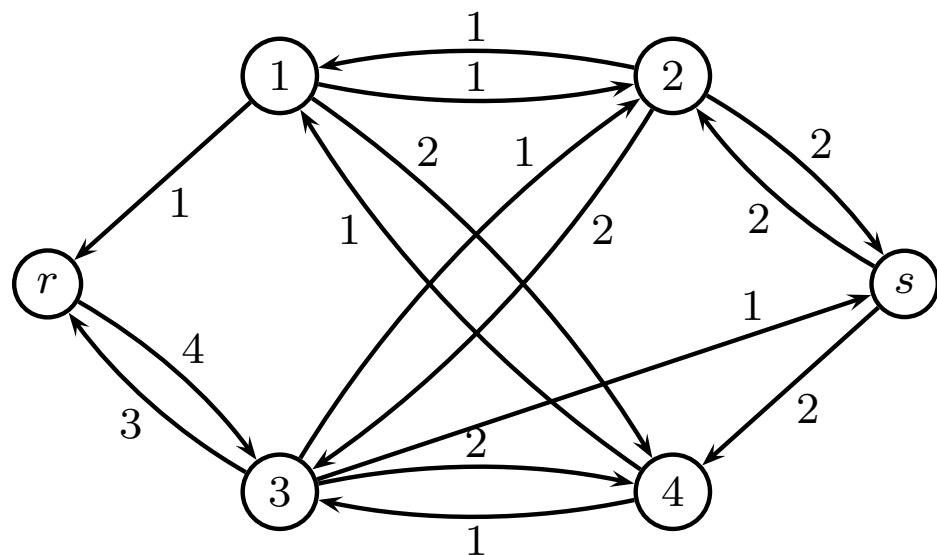
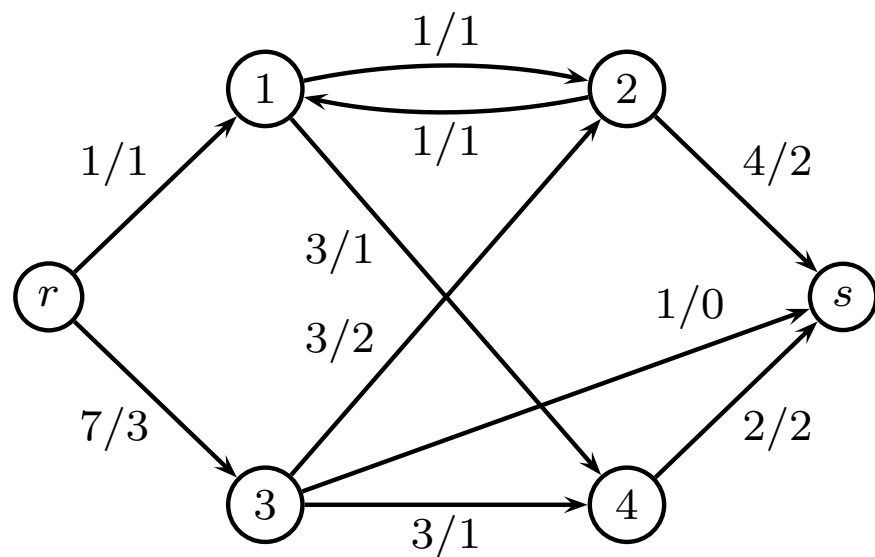
$$\begin{aligned} V(G_x) &= V \\ E(G_x) &= \{(i, j) : (i, j) \in E \wedge x_{ij} < u_{ij}\} \cup \\ &\quad \{(j, i) : (i, j) \in E \wedge x_{ij} > 0\} \end{aligned}$$

Short we write $V_x = V(G_x)$ and $E_x = E(G_x)$

- Some textbooks use the word “auxiliary graph” instead of residual graph.

The Residual graph

Graph with flow of 4, and corresponding residual graph

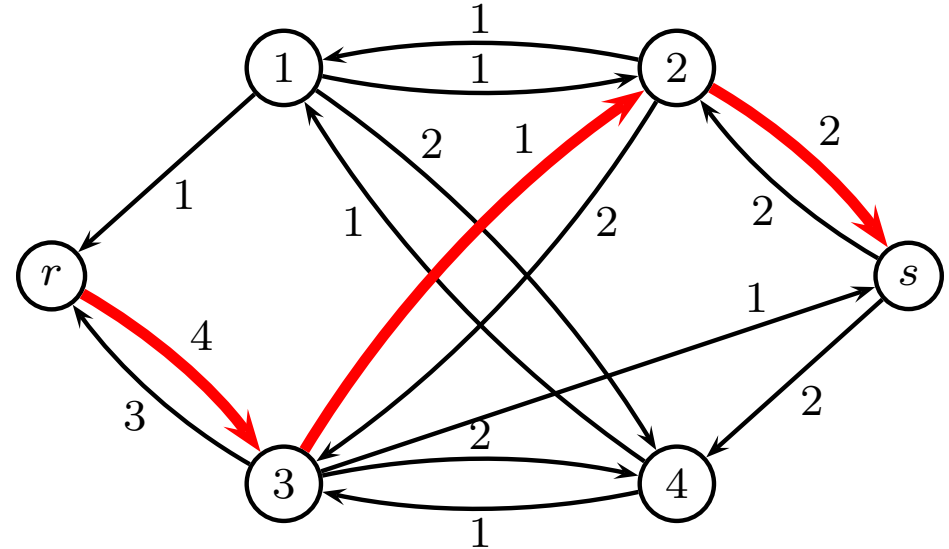
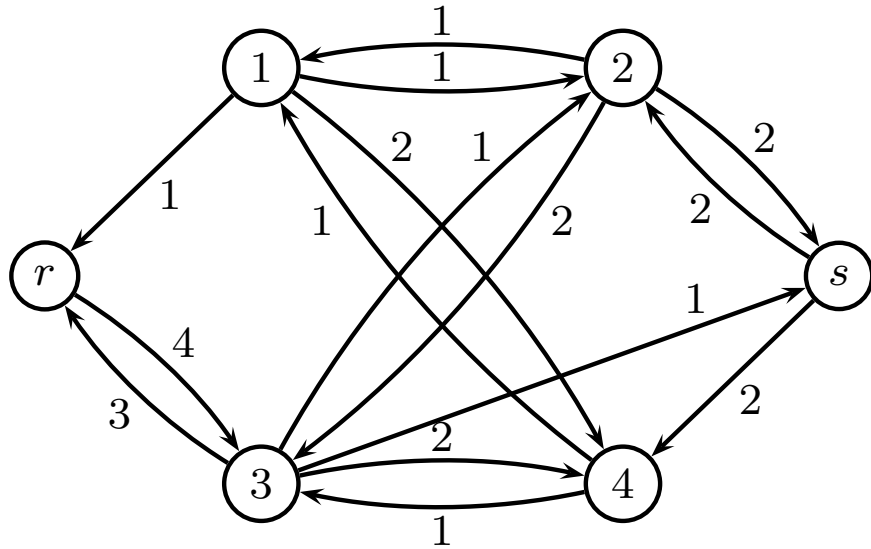


Incrementing and augmenting path

- A path is called *x*-**incrementing** if for every forward arc e $x_e < u_e$ and for every backward arc $x_e > 0$.
- A *x*-incrementing path from r to s is called *x*-**augmenting**.
- Note: this relates to the original graph. In the Residual graph all arcs in a path are forward arcs.

Augmenting path

Residual graph and augmenting path of capacity 1



Basic Approach

1. Build residual graph
2. Find augmenting path
3. If no augmenting path **stop**
4. Find capacity of augmenting path
5. Increase flow along the augmenting path.
6. Go to step 1

The Augmenting Path Algorithm

```
1  $x_{ij} \leftarrow 0$  for all  $(i, j) \in E$ ;  
2 repeat  
3   | construct residual graph  $G_x$ ;  
4   | find an augmenting path  $P$ ;  
5   | if  $s$  is reached then  
6   |   | determine max amount  $K$  of augmentation;  
7   |   | augment  $x$  along the path  $P$  by  $K$ ;  
8 until  $s$  is not reachable from  $r$  in  $G_x$ ;
```

Finding an augmenting path I

- **Input:** The network $G = (V, E, u)$ and the current feasible flow x .
- **Output:** An augmenting path from r to s and the capacity for the flow augmentation.

Initialize: all vertices are unlabeled;

$Q \leftarrow \{r\}, S \leftarrow \emptyset, p[\cdot] \leftarrow 0;$

$u_{\max}[i] \leftarrow +\infty$ for $i \in G$

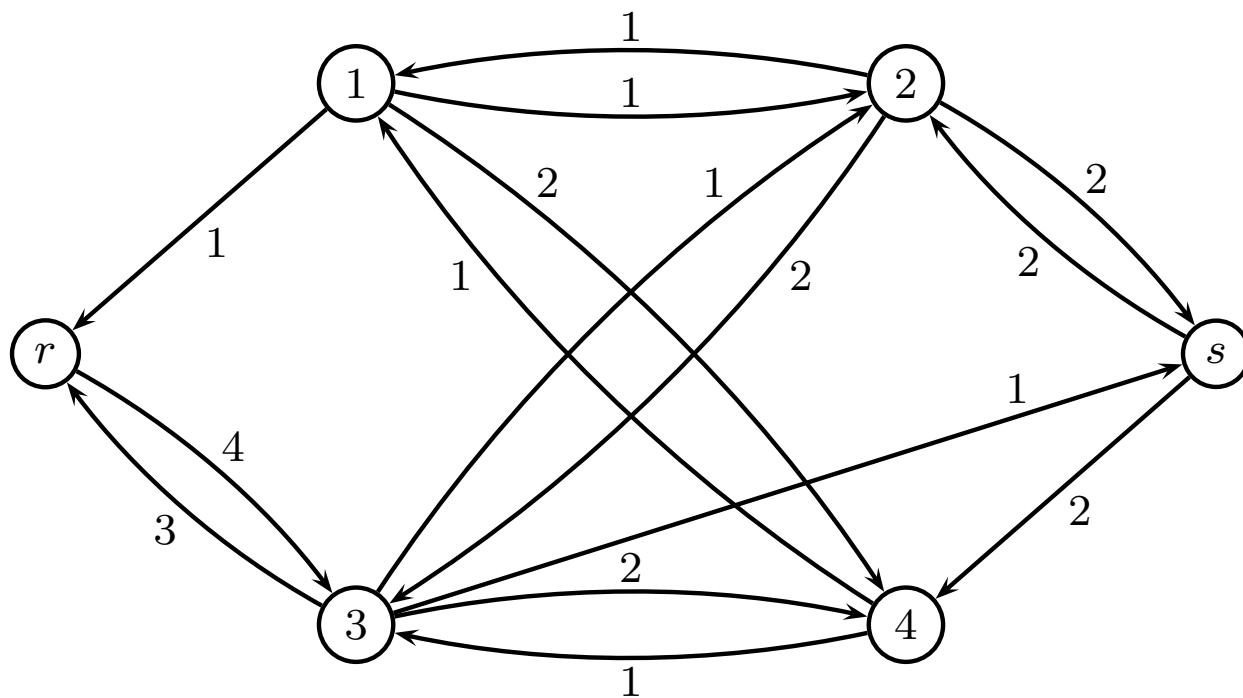
Finding an augmenting path II

```
1 while  $Q \neq \emptyset$  and  $s$  is not labeled yet do
2   select a  $i \in Q$  and scan  $(i, j) \in E_x$ ;
3   if  $(i, j) \in E$  and  $j$  unlabeled then
4     label  $j$ ;  $Q \leftarrow Q \cup \{j\}$ ;  $p[j] \leftarrow i$ ;
5      $u_{\max}[j] \leftarrow \min\{u_{\max}[i], u_{ij} - x_{ij}\}$ ;
6   if  $(j, i) \in E$  and  $j$  unlabeled then
7     label  $j$ ;  $Q \leftarrow Q \cup \{j\}$ ;  $p[j] \leftarrow i$ ;
8      $u_{\max}[j] \leftarrow \min\{u_{\max}[i], x_{ji}\}$ ;
9    $Q \leftarrow Q \setminus \{i\}$ ;  $S \leftarrow S \cup \{i\}$ ;
```

If s is labeled then the augmenting path is given by the $p[\cdot]$.
Otherwise no augmenting path exists.

Example

Residual graph

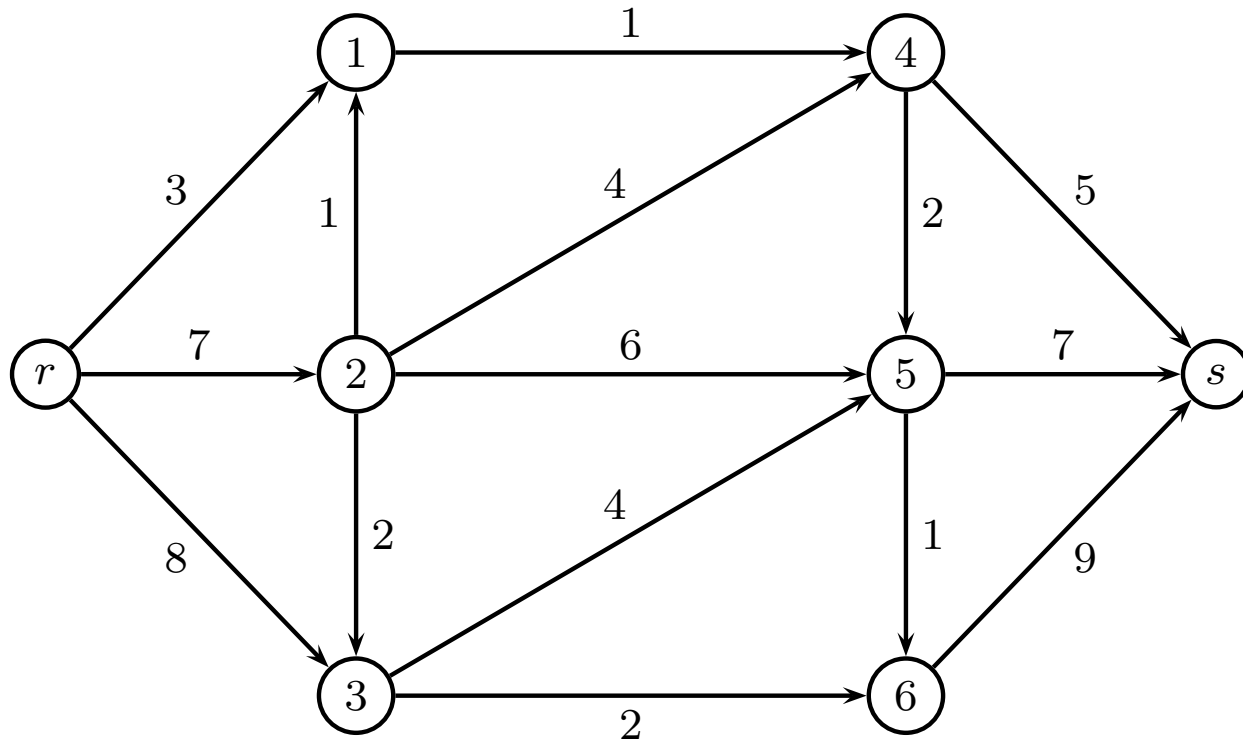


Augmenting a feasible flow x

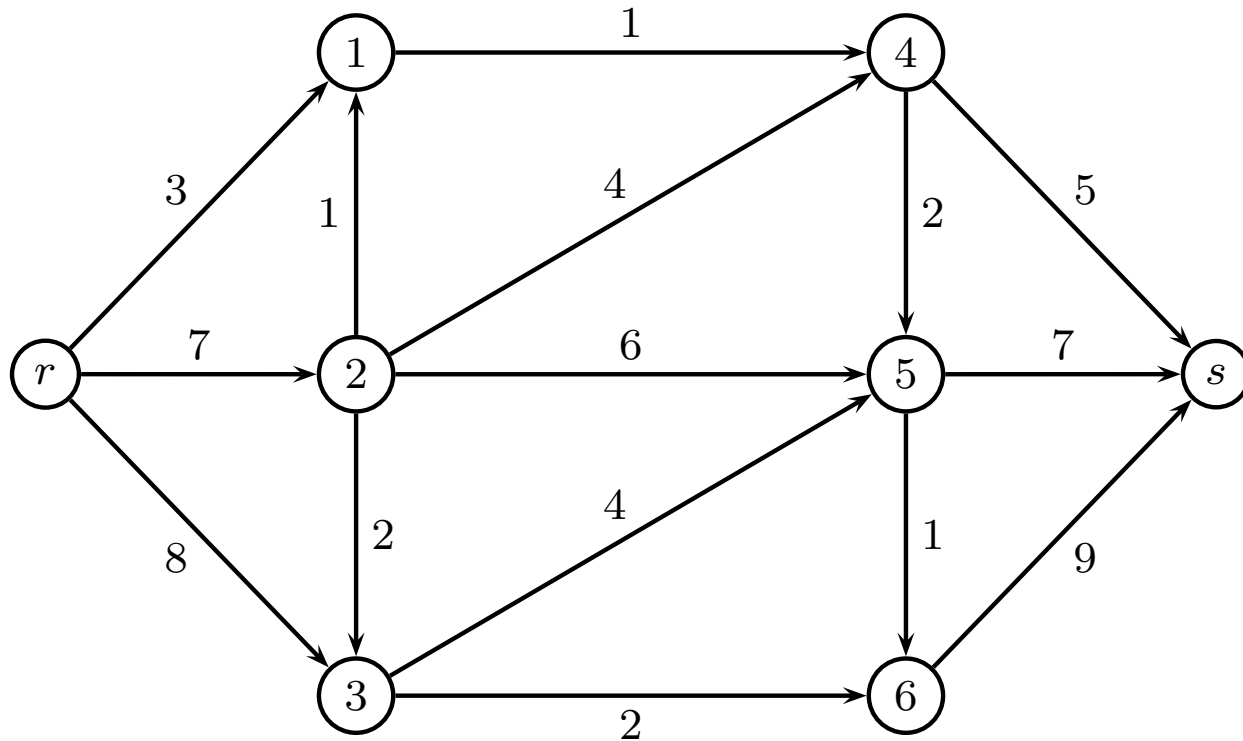
- **Input:** The network $G = (V, E, u)$, the current feasible flow x , an augmenting path P from r to s in G_x and the x -width of P .
- **Output:** An “updated” feasible flow x' .
- An updated flow is achieved by:

for $(i, j) \in P$ $(i, j) \in E : x'_{ij} \leftarrow x_{ij} + u_{\max}[s]$
for $(i, j) \in P$ $(j, i) \in E : x'_{ji} \leftarrow x_{ji} - u_{\max}[s]$
for all other $(i, j) \in E : x'_{ij} \leftarrow x_{ij}$

Example



Example



path $r25s$, capacity 6

path $r3524s$, capacity 4

path $r36s$, capacity 2

path $r14s$, capacity 1

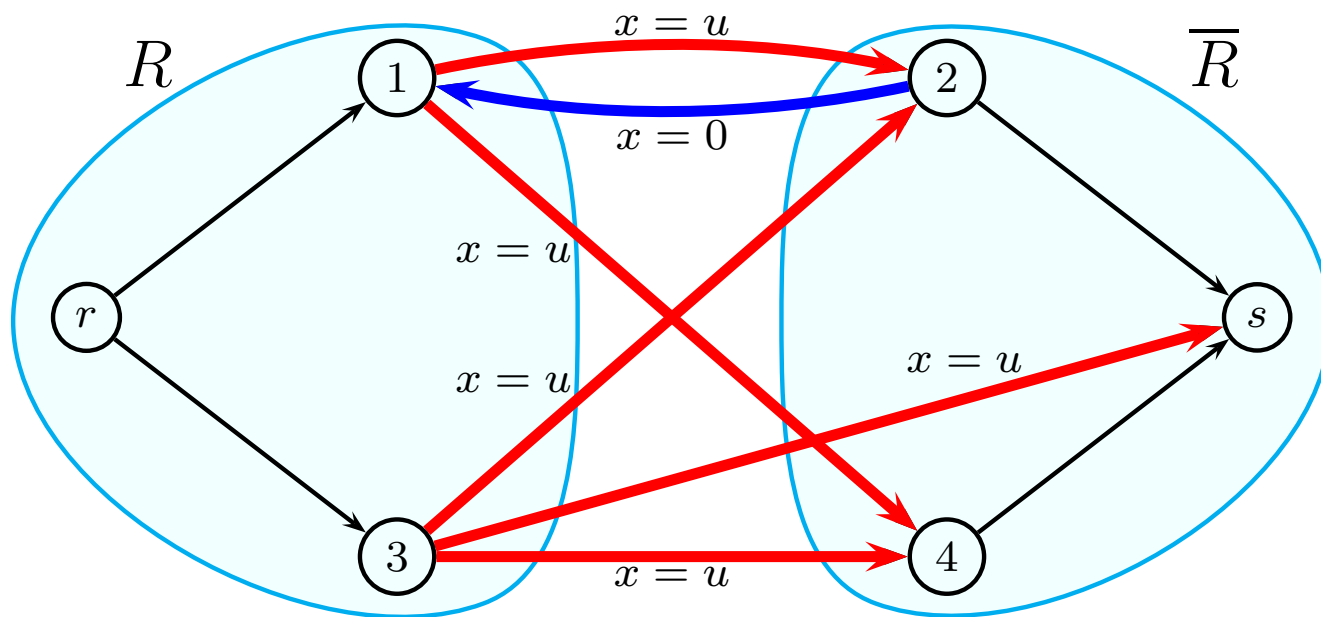
path $r25s$, capacity 1

cut $R = \{r, 1, 3\}$

Max Flow - Min Cut Theorem

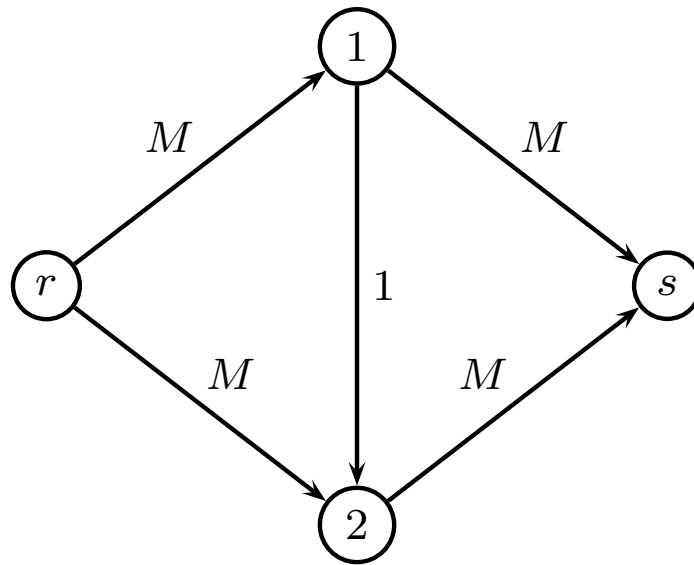
Given a network $G = (V, E, u)$ and a current feasible flow x . The following 3 statements are equivalent:

1. x is a maximum flow in G .
2. A flow augmenting path does not exist (r - s -path in G_x).
3. An (r, s) -cut R exists with capacity equal to the value of x , i.e. $u(R) = f_x(s)$.



Time complexity

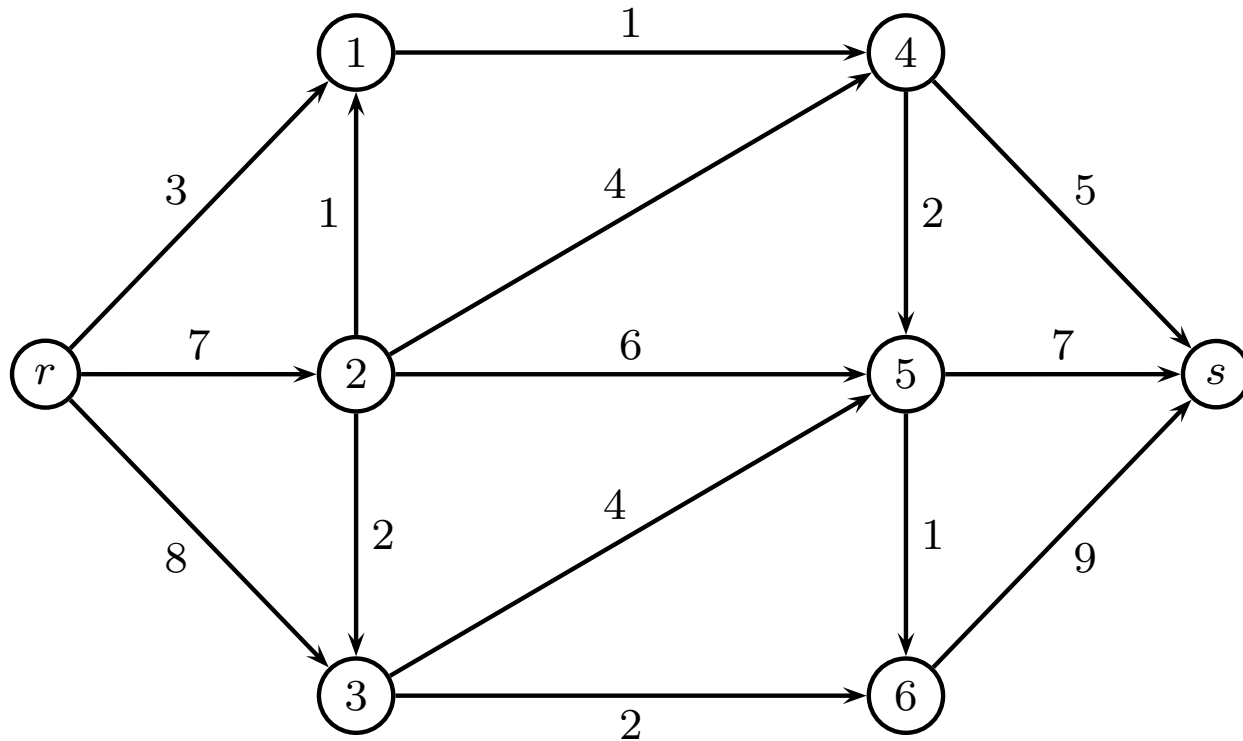
If we choose edge with capacity 1 in each iteration, we use $O(M)$ iterations.



Greedy variant

- In each iteration find the augmenting path with maximum residual capacity
- It can be shown that after $2m$ iterations the algorithm has found a maximum flow, or reduced residual capacity of max capacity augmenting path by a factor at least 2.
- At most $O(m \log U)$ iterations
- How do we find maximum capacity augmenting path?
- Each iteration $O(m)$
- Total running time $O(m^2 \log U)$.

Example



Scaling algorithm

Start with $\Delta = 2^{\log U}$. Repeat

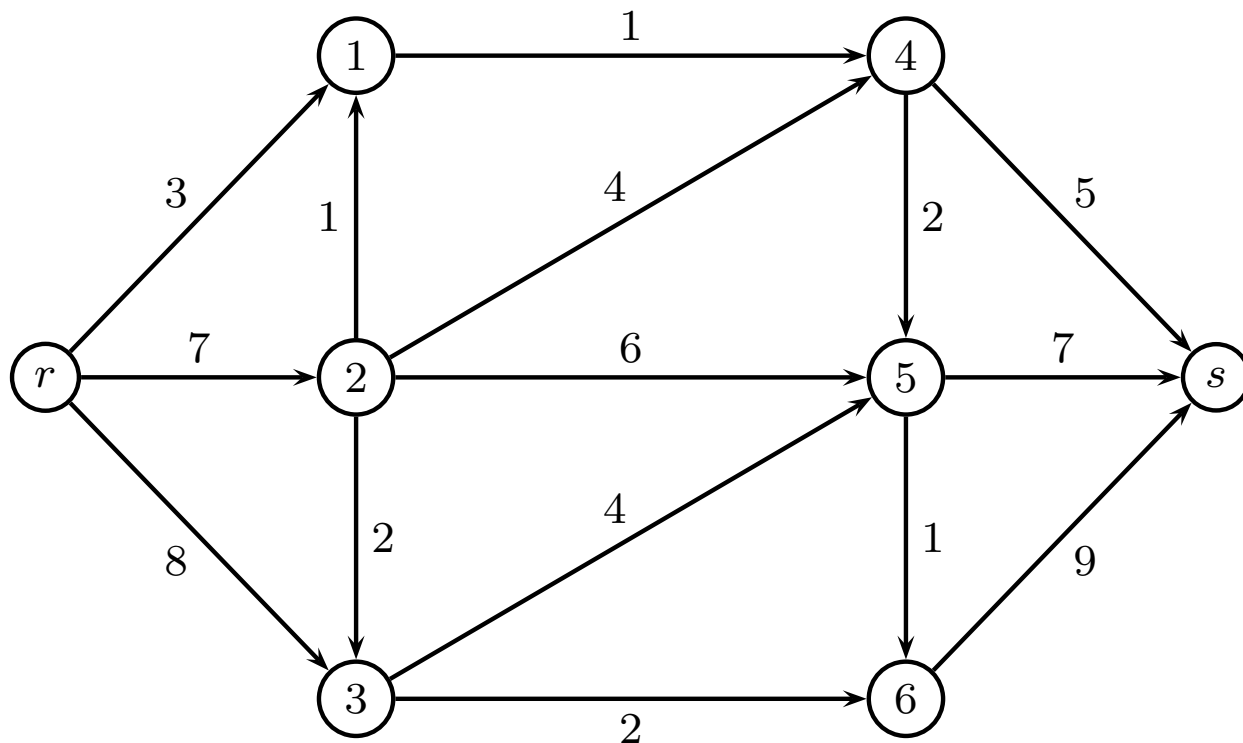
- Scaling phase: Remove all edges with residual capacity below Δ
- Solve maximum flow problem
- $\Delta := \Delta/2$

Since $\Delta = 1$ in last iteration, we find optimal solution

- It can be shown that at most $2m$ iterations for each Δ
- $O(\log U)$ scaling phases
- Running time $O(m^2 \log U)$

Example

Scaling with $\Delta = 4$



Edmonds-Karp variant

- We call an augmenting path from r to s **shortest** if it has the minimum possible number of arcs.
- The augmenting path algorithm with a breadth-first search solves the maximum flow problem in $O(nm^2)$ (Edmonds-Karp, Dinic)
- Breadth-first can easily be established using a queue. (A label correcting algorithm).

Algorithms

Linear programming	??
Ford-Fulkerson algorithm	$O(E \max f)$
Edmonds-Karp algorithm	$O(V E^2)$
Dinitz blocking flow algorithm	$O(V^2 E)$
General push-relabel maximum flow algorithm	$O(V^2 E)$
Push-relabel algorithm with FIFO vertex selection rule	$O(V^3)$
Dinitz blocking flow algorithm with dynamic trees	$O(V E \log(V))$
Push-relabel algorithm with dynamic trees	$O(V E \log(V^2 / E))$
MPM (Malhotra, Pramodh-Kumar and Maheshwari) algorithm	$O(V^3)$
Jim Orlin's + KRT (King, Rao, Tarjan)'s algorithm	$O(V E)$

Modeling

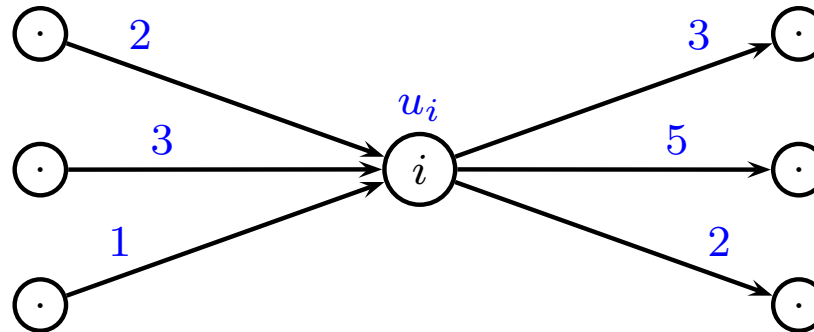
- Capacity on nodes
- Multiple roots and sinks
- Lower bound on edges (important in B&B)
- Circulation problem
- Circulation problem with upper/lower bounds on edges
- Maximum flow with upper/lower bound on edges

Max-flow, capacity on nodes

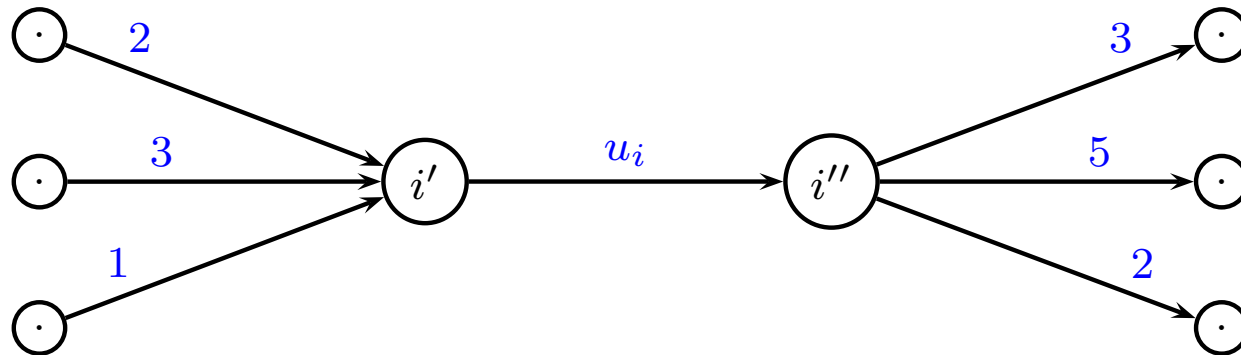
Assume that there is a capacity u_i on node i

- Replace node i with two nodes i' and i''
- Create an arc from i' to i'' with capacity u_i
- Replace every ingoing arc (j, i) with an arc (j, i')
- Replace every outgoing arc (i, j) with an arc (i'', j)

Max-flow, capacity on nodes



becomes



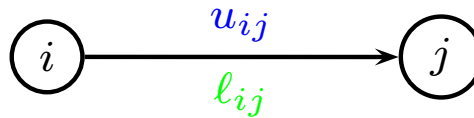
Max-flow, multiple roots and sinks

Max-flow must have *one* root and sink.
(otherwise augmenting path not well-defined)

- Create super-root r^*
- Create super-sink s^*
- Connect super-root r^* to roots with infinite capacity
- Connect sinks to super-sink s^* with infinite capacity

Max-flow, lower bound on edges

Assume we have lower bound ℓ_{ij} on an edge (i, j)



Linear programming can handle lower bounds

Difficult to ensure bounds using augmenting path method
But we can do it by first studying circulation problem

Circulation problem

No objective function, just seeking feasible solution
Multiple roots and sinks, with demand

If a node has positive demand $d_i > 0$ it is a *sink*

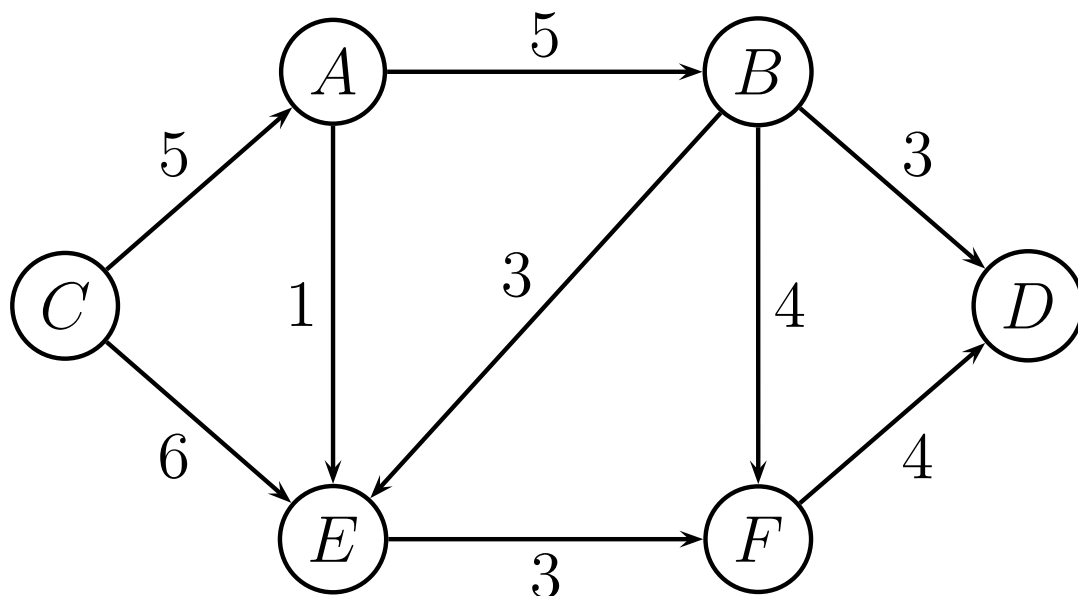
If a node has negative demand $d_i < 0$ it is a *source*

Obviously, must have $\sum_{i \in V} d_i = 0$

Transformation to ordinary max-flow

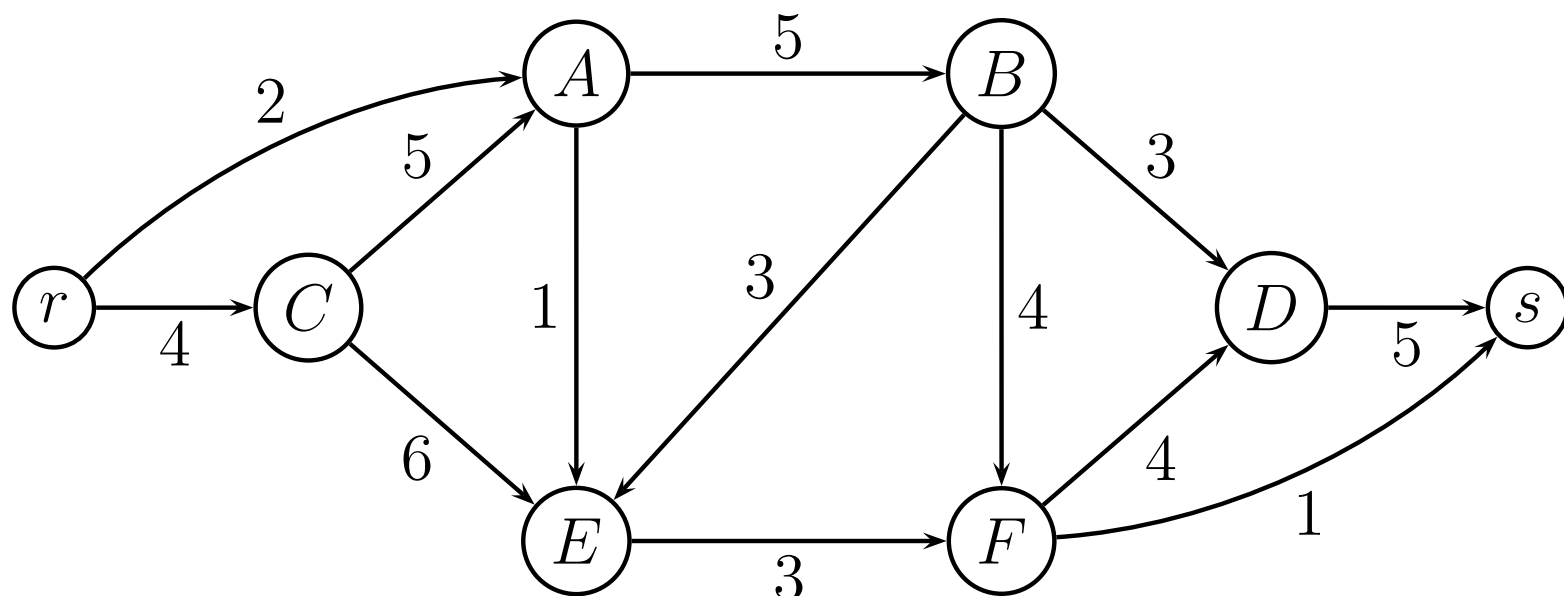
- Create super-root r^*
- Create super-sink s^*
- Connect super-root r^* to sources i with capacity $-d_i$
- Connect sinks i to super-sink s^* with capacity d_i

Circulation problem, example



node i	A	B	C	D	E	F
demand d_i	-2	0	-4	5	0	1

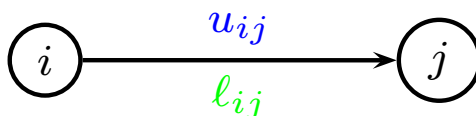
Circulation problem, example



If a max-flow solution of value $2 + 4$ exists, circulation problem is *feasible*
(all edges from r need to be filled)

Circulation, lower bound on edges

Assume lower bound ℓ_{ij} , capacity u_{ij} on edge (i, j)



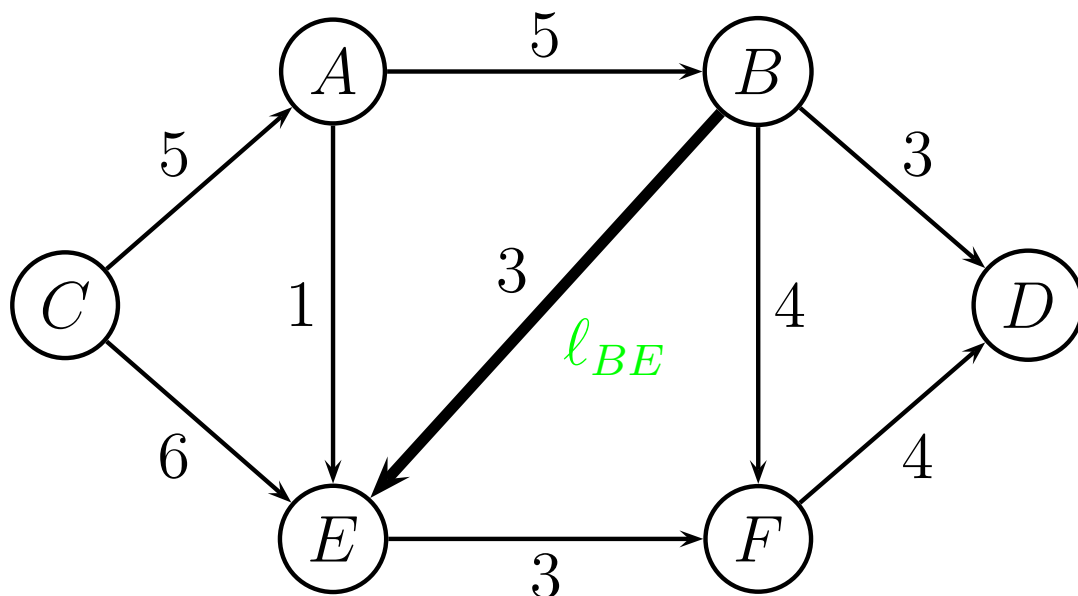
- Decrease capacity $u_{ij} := u_{ij} - \ell_{ij}$
- Increase demand $d_i := d_i + \ell_{ij}$
- Decrease demand $d_j := d_j - \ell_{ij}$

Next, use previous slide for circulation problem

After the circulation problem is solved, add ℓ_{ij} to x_{ij}

Circulation, lower bound, example

Lower bound on edge (B, E) of value $\ell_{BE} = 2$.



node i	A	B	C	D	E	F
demand d_i	-2	0	-4	5	0	1

Circulation, lower bound, example

Lower bound on edge (B, E) of value $\ell_{BE} = 2$.

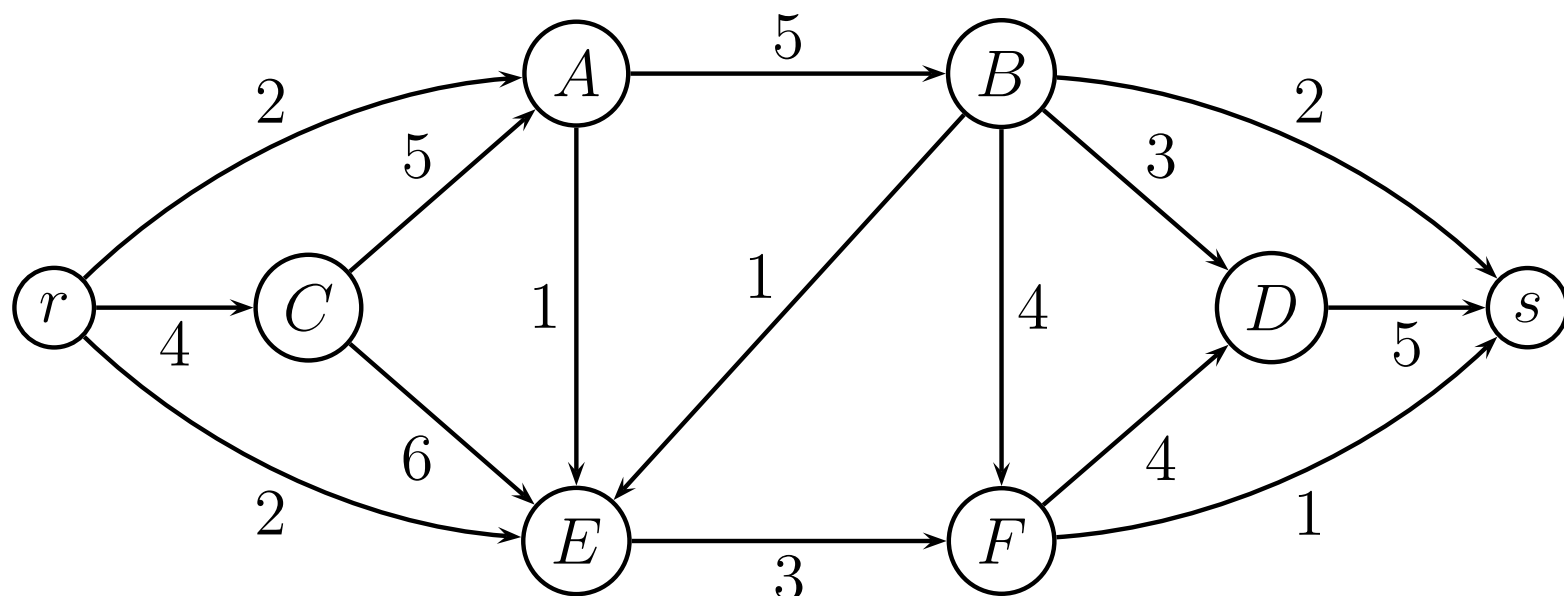
- Reduce the capacity of edge (B, E) from 3 to 1.
- Increase demand at node B from 0 to 2.
- Decrease demand at node E from 0 to -2.

node i	A	B	C	D	E	F
demand d_i	-2	2	-4	5	-2	1

Next, formulate as ordinary max-flow problem

After the circulation problem is solved, add ℓ_{BE} to x_{BE}

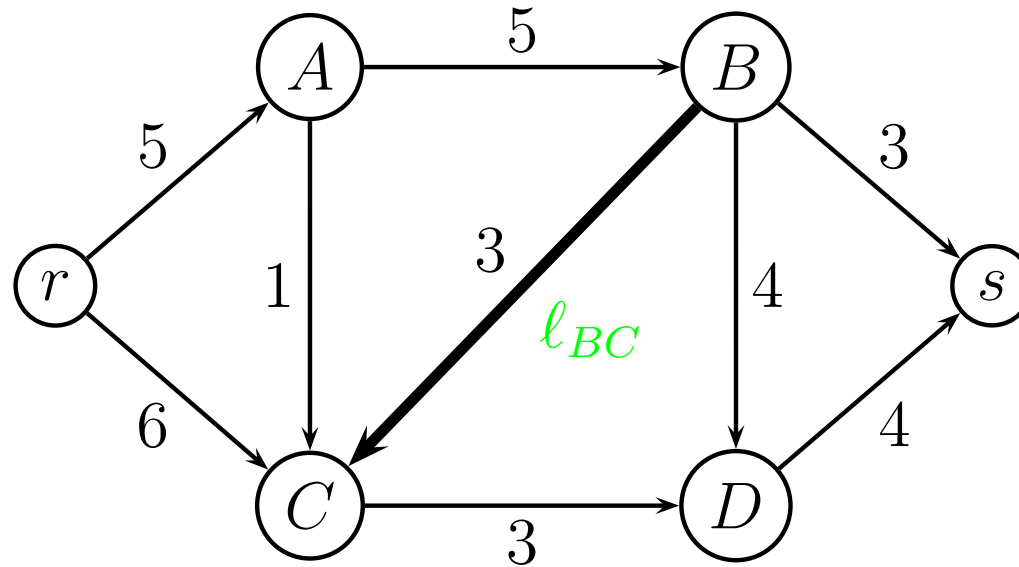
Circulation, lower bound, example



If a max-flow solution of value $2 + 4 + 2$ exists, circulation problem is *feasible*
(all edges from r need to be filled)

Maximum flow with lower bound(s)

Example: Lower bound on edge (B, C)



- Guess optimal flow z . Then we get circulation problem

node i	r	s	A	B	C	D
demand d_i	$-z$	z	0	0	0	0

- Transform circulation problem with lower bounds to ordinary max-flow problem
- Use binary search to find correct value of z

Maximum flow with lower bound(s)

Running time

- Let u be an upper bound on maximum flow problem (e.g. sum of capacities from r)
- We need to solve a maximum-flow problem $\log u$ times
- This is a polynomial algorithm